

Web application and API security assessment

Prepared for Sample Retail Private Limited

REPORT REFERENCE

ATT-2026-000

VERSION

1.0

ISSUED

20 July 2026

ASSESSMENT PERIOD

13 July 2026 to 17 July 2026

ASSESSMENT TYPE

Grey box

SEVERITY MODEL

CVSS 4.0

Contents

Document control

1. Executive summary
2. Risk overview
3. Scope
4. Methodology
5. Coverage matrix
6. Attack narrative
7. Detailed findings
8. Positive observations
9. Strategic recommendations
10. Compliance mapping
11. Appendices
12. Limitations and disclaimer

Document control

VERSION	DATE	CHANGE
0.1	18 July 2026	Internal draft for review
1.0	20 July 2026	Issued to the client

Responsibility

Author	Attestor Security
Reviewer	Attestor Security
Distribution	Sample Retail Private Limited — engineering and security

Handling

This document describes unfixed security weaknesses in systems belonging to Sample Retail Private Limited. Keep it in access-controlled storage. Share it only with people who need it. Do not publish or quote from it without written consent.

1. Executive summary

Sample Retail asked us to assess the customer web application and the API behind it before the July release. We tested for five working days across three user roles against the staging environment running the release candidate, using the published OWASP methodology and the tools listed in section 4.

The application is not ready to ship. Three findings, taken together, give an unauthenticated attacker the whole customer database in under an hour, and they do it through ordinary application features rather than anything exotic. Section 6 sets that path out step by step. Separately, and independently of that chain, the checkout accepts the order total from the browser, which means an order can be placed for any amount the customer chooses.

Not everything is bad, and the parts that are right are right for structural reasons rather than by accident. Password storage uses a modern hashing function with per-user salts. TLS is correctly configured. Card details never reach the application because the payment integration uses hosted fields — a design decision that removed an entire class of finding from this assessment and is worth carrying into the next feature.

The remediation order in section 9 is not the severity order. Two of the findings that are individually low or medium are steps in the chain, and each of them is a small change.

The first things to do

- 1. Parameterise the login query (ATT-2026-000-001). One change to one handler, and it removes the most direct route to administrator.
- 2. Pin the accepted signature algorithm on token verification and rotate the signing key (ATT-2026-000-003). Until this is done, every other authentication control is advisory.
- 3. Recompute the order total on the server (ATT-2026-000-006). This one is losing money now rather than hypothetically.

2. Risk overview

Findings by severity

CRITICAL	2	
HIGH	4	
MEDIUM	3	
LOW	1	
INFORMATIONAL	0	

Findings by category

CATEGORY	FINDINGS
A07:2025	3

CATEGORY	FINDINGS
A05:2025	2
A01:2025	2
A06:2025	1
A02:2025	1
A03:2025	1

Findings by asset

ASSET	FINDINGS
juice.attestor-lab.internal	10

Fix things in the order given in section 9, which is not severity order. Some low-severity findings are steps in the chain shown in section 7 and are cheap to close.

3. Scope

Assets in scope

- juice.attestor-lab.internal – the customer web application and its REST API
- The administration interface at the same origin
- Three user roles: anonymous visitor, registered customer, administrator

Explicitly out of scope

- The payment provider, which is a third-party service outside the client’s authorisation
- The corporate network, staff email and workstations
- Denial-of-service, load and volumetric testing, which are excluded by agreement and are never performed
- Social engineering and physical access

Environments

- Staging, running the release candidate for the 2026-07 production deployment

Roles and accounts used

- anonymous – no session
- customer – two separate accounts, to test between customers as well as between roles
- administrator – one account

Test dates

13 July 2026 to 17 July 2026, Asia/Kolkata. Assessment type: Grey box.

Constraints imposed by the client

- Testing was permitted between 04:00 and 13:00 UTC on working days, at the client’s request.
- Outbound requests were limited to 4 per second per host throughout.

4. Methodology

Testing followed OWASP Web Security Testing Guide 4.2, OWASP Application Security Verification Standard 5.0, OWASP Top 10:2025, OWASP API Security Top 10:2023, NIST SP 800-115, PTES.

The work ran in four phases. First, discovery and enumeration without credentials. Second, automated scanning within the agreed rate limits. Third, a tester checked every candidate finding by hand. Fourth, a tester tested access control, business logic and multi-step flows directly. Tools were used for coverage, not for conclusions. A tester reproduced every finding in this report. Anything a tool reported that could not be reproduced was left out.

Tools used

TOOL	VERSION	USED FOR
OWASP ZAP	2.16.1	Authenticated crawling and active scanning
nuclei	3.4.7	Templated checks for known issues and exposures
sqlmap	1.9.4	Injection confirmation, read-only settings
ffuf	2.1.0	Content discovery within the agreed rate limit
Playwright	1.56.0	Authenticated browsing and evidence capture
trivy	0.68.0	Dependency and container analysis
testssl.sh	3.2	TLS configuration review

Severity model

Severities come from CVSS 4.0. Each finding prints its vector string, so you can recompute the score yourself. Where a tester changed a computed severity because the business context changes the impact, the finding says why.

Excluded techniques

We did not run denial-of-service, stress, load or volumetric tests. We do not offer them. We did not take destructive actions, extract production personal data in bulk, social-engineer staff, or attempt physical entry.

5. Coverage matrix

This matrix is built from what actually ran. A check is marked tested only where a completed run or a recorded manual test covered it. Everything else carries the reason it does not. **Not present** means the application has nothing for the check to test, such as no file upload, and is not a gap in the testing.

MODULE	CHECKS	TESTED	PARTLY	NOT TESTED	NOT PRESENT
Reconnaissance	19	17	0	2	0
Web application	108	80	0	28	0
API	62	51	0	11	0

Not part of this engagement: Mobile, Cloud, Source code, Network, LLM application. Their checks are not listed below.

Information gathering

TESTED, NOTHING FOUND (11): Application entry point and execution path mapping; Webserver metafile review; Subdomain and hostname enumeration; DNS record review; Subdomain takeover; HTTP service discovery and fingerprinting; Content and path discovery; JavaScript endpoint and secret extraction; Virtual host discovery; Technology and version inventory; Attack surface change detection.

NOT TESTED (2) — A manual check. No tester recorded doing it in this engagement. Search engine and public source discovery; Application architecture mapping.

Network and infrastructure

TESTED, NOTHING FOUND (1): Port and service enumeration.

NOT TESTED (1) — No proxy or CDN sits in front of the staging environment, so the class does not apply here. HTTP request smuggling and desynchronisation.

Cryptography

TESTED, NOTHING FOUND (5): TLS configuration and certificate review; Sensitive data exposure in transit; Weak cryptographic storage; Padding oracle and ciphertext manipulation; Use of predictable randomness.

Configuration and deployment

CHECK	STATE	NOTE	FINDINGS
HTTP security header configuration	TESTED		1
Rate limiting on sensitive endpoints	TESTED		1

TESTED, NOTHING FOUND (10): SPF, DKIM and DMARC review; Exposed version control and build artefacts; CORS policy review; HTTP method handling; File extension and backup file handling; Administrative interface exposure; Caching of authenticated responses; Host header handling; Application platform configuration; API CORS and pre-flight consistency.

NOT TESTED (2) — A manual check. No tester recorded doing it in this engagement. Path confusion and route resolution; File and directory permissions on the host.

Cloud

TESTED, NOTHING FOUND (1): Exposed cloud storage discovery.

Supply chain and code

TESTED, NOTHING FOUND (2): Credentials exposed in public sources; Third-party script inclusion and subresource integrity.

Session management

TESTED, NOTHING FOUND (6): Cookie attribute review; Session token generation and entropy; Session fixation; Cross-site request forgery; Logout and session-riding side effects; Session identifiers and sensitive values in URLs.

NOT TESTED (4) — A manual check. No tester recorded doing it in this engagement. Session timeout and termination; Concurrent session handling; Session variable overloading; Session hijacking exposure.

Authentication

CHECK	STATE	NOTE	FINDINGS
Username and account enumeration	TESTED		1
JWT and self-contained token handling	TESTED		1

TESTED, NOTHING FOUND (6): Default and weak credentials on exposed components; Registration process review; Credentials transmitted over an encrypted channel; Authentication bypass; API authentication mechanisms; Token lifecycle and revocation.

NOT TESTED (10) — A manual check. No tester recorded doing it in this engagement. Password policy and credential strength; Brute force and credential stuffing protection; Multi-factor authentication implementation; Password reset and recovery flow; Remember-me and persistent login; OAuth and OpenID Connect flow review; SSO and federated identity handling; Re-authentication for sensitive operations; Username policy and account identifier handling; Authentication on alternative channels.

Access control

CHECK	STATE	NOTE	FINDINGS
Horizontal access control between users	TESTED		1
Path traversal and local file access	TESTED		1

TESTED, NOTHING FOUND (11): Vertical access control across roles; Tenant isolation in multi-tenant applications; Forced browsing to unlinked functionality; Privilege escalation through request parameters; Server-side request forgery; Uploaded file access control; Broken object level authorisation; Broken function level authorisation; Object property level authorisation; GraphQL field and node authorisation; Authentication and authorisation of the model endpoint.

NOT TESTED (2) — A manual check. No tester recorded doing it in this engagement. Account provisioning and de-provisioning; Role definitions and the permission model.

Input validation

CHECK	STATE	NOTE	FINDINGS
SQL injection	TESTED		1
Stored cross-site scripting	TESTED		1

TESTED, NOTHING FOUND (18): NoSQL and ORM injection; OS command injection; Reflected cross-site scripting; Server-side template injection; XML external entity processing; Insecure deserialisation; File upload handling; Open redirection; HTTP header and response splitting; LDAP and directory injection; Mass assignment and object property binding; CSV and spreadsheet formula injection; HTTP parameter pollution; Server-side include injection; XPath injection; Format string injection; Schema-driven injection and type confusion; Content type and parser handling.

NOT TESTED (2) — A manual check. No tester recorded doing it in this engagement. IMAP and SMTP injection through mail features; Incubated and delayed-execution vulnerabilities.

Client-side

TESTED, NOTHING FOUND (9): DOM-based cross-site scripting; Prototype pollution; Clickjacking and UI redress; WebSocket security; Source maps and build artefacts in production; Reverse tabnabbing on outbound links; HTML injection; Client-side JavaScript execution from controllable input; Client-side resource manipulation.

NOT TESTED (3) — A manual check. No tester recorded doing it in this engagement. Cross-origin messaging; Client-side storage of sensitive data; CSS injection.

Business logic

CHECK	STATE	NOTE	FINDINGS
Price, quantity and discount manipulation	TESTED		1

TESTED, NOTHING FOUND (9): Workflow and process bypass; Coupon and promotion abuse; Race conditions on limited resources; Function and transaction limits; Integrity checks on submitted data; Automated abuse of sensitive business flows; Chaining of low-severity findings; Process timing and ordering; Payment flow integrity.

Error handling

TESTED, NOTHING FOUND (2): Error handling and information disclosure; API error verbosity.

NOT TESTED (2) — A manual check. No tester recorded doing it in this engagement. Behaviour under failure of a dependency; Security event logging and alerting.

API

TESTED, NOTHING FOUND (9): API specification import and coverage; Endpoint inventory, versioning and deprecated routes; Excessive data exposure in responses; Unrestricted resource consumption; API rate limiting and quota enforcement; GraphQL introspection and schema exposure; GraphQL query depth, aliasing and batching abuse; Webhook delivery and receipt security; Idempotency and request replay.

NOT TESTED (3) — A manual check. No tester recorded doing it in this engagement. Unsafe consumption of third-party APIs; gRPC and binary protocol handling; Mobile backend parity with the web API.

Mobile

NOT TESTED (1) — A manual check. No tester recorded doing it in this engagement. Backend traffic capture and analysis.

LLM and AI

NOT TESTED (1) — A manual check. No tester recorded doing it in this engagement. Model output rendered as HTML.

6. Attack narrative

From an anonymous visitor to every customer record in four steps

```

anonymous visitor
  |
  | 1. reset endpoint distinguishes registered addresses      (ATT-2026-000-008, low)
  v
list of real customer addresses
  |
  | 2. download endpoint returns the configuration file      (ATT-2026-000-005, high)
  v
token signing material
  |
  | 3. verifier trusts the algorithm named in the token      (ATT-2026-000-003, critical)
  v
valid session for any account, including administrator
  |
  | 4. ordinary administrative screens
  v
every order, address and payment reference

```

Step 1 — Confirm which addresses are customers

The password reset endpoint distinguishes registered from unregistered addresses (ATT-2026-000-008). A list of 5,000 candidate addresses was reduced to the subset that exists on the platform. On its own this is a low-severity finding and would reasonably be scheduled for a later sprint.

Step 2 — Read the application configuration

The download endpoint resolves a client-supplied filename without normalising it (ATT-2026-000-005), which returns the configuration file. That file contains the material used to sign session tokens.

Step 3 — Become any of those customers

With the signing material, and with the verifier accepting the algorithm named in the token header (ATT-2026-000-003), a valid session token can be produced for any of the addresses confirmed in step 1 — including the administrator.

Step 4 — Take the data

The administrator session reaches every order, address and payment reference. No separate exploitation was required: the ordinary administrative screens do it. During testing this was confirmed on our own test accounts only, and no customer data was read or retained.

Each of the first two steps would be triaged as low or high on its own and would sit in a backlog. Together with the token weakness they produce an unauthenticated path to the complete customer database in under an hour. This is why the remediation order in this report is not simply the severity order.

7. Detailed findings

Ordered by severity, then by reference. A tester reproduced every finding here.

ATT-2026-000-001

Login form allows authentication as any user through SQL injection

CRITICAL

9.3

CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

OWASP High

CWE-89

A05:2025

WSTG-INPV-05

ASVS 1.2.1

AFFECTED

- POST `juice.attestor-lab.internal/rest/user/login` (email)

BUSINESS IMPACT

An unauthenticated attacker becomes the administrator in a single request. From that account every customer record, order and payment reference in the application is readable, and product and pricing data is writable. There is no rate limit and no logging that would distinguish this from an ordinary failed login, so the first indication of compromise would be the consequences.

LIKELIHOOD

High. The payload is in every introductory tutorial for this class of issue, the endpoint is public, and no authentication or prior knowledge is required.

ATTACKER PREREQUISITES

None. Network access to the login page is sufficient.

TECHNICAL DESCRIPTION

The email field on the login form is concatenated into a SQL statement without parameterisation. A payload that terminates the string literal and comments out the remainder of the statement authenticates the request as the first row of the users table, which is the administrator. No password is required. The same parameter also supports UNION-based extraction of arbitrary tables, which was confirmed on the schema only and not used to read customer data.

REPRODUCTION

1. Open the login page at `https://juice.attestor-lab.internal/#/login`.
2. Enter the email address value: `' OR 1=1--`
3. Enter any value in the password field.
4. Submit the form.
5. Observe that the response contains a valid authentication token and the session is that of the administrator account.

EVIDENCE

AUTHENTICATION REQUEST AND RESPONSE, MASKED

```

POST /rest/user/login HTTP/1.1
Host: juice.attestor-lab.internal
Content-Type: application/json

{"email":"' OR 1=1--","password":"anything"}

--- response ---
HTTP 200
Content-Type: application/json

{"authentication":{"token":"[REDACTED]","bid":1,"umail":"a*****@j*****.***"}}

```

REMEDIATION

Replace the concatenated statement in the login handler with a parameterised query, binding the email address as a parameter rather than interpolating it. In the Sequelize models used by this application that means replacing the raw `sequelize.query` call with a `findOne` using a `where` clause, or passing `replacements` with named bind parameters. Parameterisation is the fix; input filtering is not, because the same data has to be accepted as a legitimate email address. Rotate the administrator credential afterwards, and review the authentication logs for the period the issue was live.

REFERENCES

- [OWASP Cheat Sheet: SQL Injection Prevention](https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html)
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [CWE-89: SQL Injection](https://cwe.mitre.org/data/definitions/89.html)
<https://cwe.mitre.org/data/definitions/89.html>

OWASP RISK RATING

Likelihood 7.00 (high), impact 5.50 (medium), giving an overall rating of **High**. Scored against the OWASP Risk Rating Methodology; each factor below is a published option of that method.

FACTOR	SCORE
Skills required	3
Motive	9
Opportunity	9
Population size	9
Ease of discovery	7
Ease of exploit	5
Awareness	6
Intrusion detection	8
Loss of confidentiality	7
Loss of integrity	5
Loss of availability	1
Loss of accountability	7

FACTOR	SCORE
Financial damage	7
Reputation damage	5
Non-compliance	5
Privacy violation	7

ATT-2026-000-003

Session tokens accepted with the signature algorithm set to none

CRITICAL	9.3	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N	CWE-347	A07:2025	API2:2023
WSTG-SESS-10	ASVS 9.1.1				

AFFECTED

- juice.attestor-lab.internal/rest/* (Authorization)

BUSINESS IMPACT

Authentication is effectively optional. Every control that depends on identity – order history, address book, payment methods, the administration area – can be reached by constructing a token. Because the forged token is structurally valid, nothing in the application logs distinguishes it from a legitimate session.

LIKELIHOOD

High. Publicly documented technique, no prerequisites, no rate limiting involved.

ATTACKER PREREQUISITES

None beyond network access.

TECHNICAL DESCRIPTION

The application issues RS256-signed JSON Web Tokens but the verification path accepts a token whose header declares `alg: none` and which carries no signature. A token can therefore be minted for any account, including the administrator, without possession of any key.

REPRODUCTION

- Capture a legitimate token from a normal login and decode it.
- Change the header to `{"alg":"none","typ":"JWT"}` and set the payload subject to the administrator account.
- Re-encode the header and payload, leaving the signature segment empty, and keep the trailing dot.
- Send any authenticated request with that token in the Authorization header.
- Observe a 200 response containing administrator-scoped data.

EVIDENCE

TOOL OUTPUT, REDACTED

```
$ attestor run --check ATT-2026-000-003
reproduced 1/1 attempts
evidence captured: 1 object
```

REMEDIATION

Pin the accepted algorithm explicitly at verification time rather than reading it from the token header — pass `algorithms: ["RS256"]` to the verifier — and reject any token whose signature segment is empty. Verify the issuer and audience claims at the same point. Rotate the signing key, because tokens already issued cannot be distinguished from forged ones, and invalidate existing sessions.

REFERENCES

- **OWASP Cheat Sheet: JSON Web Token for Java**
https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- **CWE-347: Improper Verification of Cryptographic Signature**
<https://cwe.mitre.org/data/definitions/347.html>

ATT-2026-000-002

Basket contents of any customer retrievable by changing the basket identifier

HIGH	8.1	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:L/VA:N/SC:N/SI:N/SA:N	CWE-639	A01:2025	API1:2023
WSTG-ATHZ-04	ASVS 8.1.2				

AFFECTED

- GET `juice.attestor-lab.internal/rest/basket/{id}`
- GET `juice.attestor-lab.internal/api/BasketItems/{id}`

BUSINESS IMPACT

Any registered customer can read every other customer's basket by counting upwards. The basket reveals purchase intent and the coupon codes in use, and the same authorisation gap on the item endpoint allows items to be added to another customer's basket. For a retailer this is both a privacy exposure and a route to manipulating another customer's order.

LIKELIHOOD

High. Registration is open, identifiers are sequential, and the request can be made with a browser and no tooling.

ATTACKER PREREQUISITES

A registered account, which anyone can create.

TECHNICAL DESCRIPTION

The basket endpoint authorises on the presence of a valid session rather than on ownership of the basket being requested. Changing the numeric identifier in the path returns another customer's basket, including the products, quantities and the coupon applied. Identifiers are sequential, so the whole set is enumerable.

REPRODUCTION

1. Register two accounts, A and B, and add a product to each basket.
2. Sign in as A and note the basket identifier in the response to GET `/rest/basket/{id}`.
3. Request the adjacent identifier, GET `/rest/basket/{id-1}`, with account A's session.
4. Observe that account B's basket contents are returned with a 200 response.

EVIDENCE

BASKET BELONGING TO A DIFFERENT CUSTOMER, RETRIEVED WITH ACCOUNT A'S SESSION

```
GET /rest/basket/2 HTTP/1.1
Host: juice.attestor-lab.internal
Authorization: [REDACTED]

--- response ---
HTTP 200

{"data":{"id":2,"UserId":2,"Products":[{"name":"Melon
Bike","price":2999,"quantity":1}], "coupon":"WELCOME10"}}
```

REMEDIATION

Authorise on ownership, not on session validity: the basket handler must load the basket and compare its owning user against the identity in the session before returning it, rejecting the request with 404 rather than 403 so the identifier space is not confirmed. Apply the same check to the BasketItems handlers, which have the same gap. Adding a non-sequential identifier is worth doing but is not the fix on its own.

REFERENCES

- OWASP Cheat Sheet: Insecure Direct Object Reference Prevention
https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html
- OWASP API Security Top 10: API1:2023 Broken Object Level Authorization
<https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>

ATT-2026-000-004

Product review field stores script that executes in every viewer's session

HIGH

7.6

CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:P/VC:H/VI:L/VA:N/SC:L/SI:L/SA:N

CWE-79

A05:2025

WSTG-INPV-02

ASVS 1.3.1

AFFECTED

- PUT juice.attestor-lab.internal/rest/products/{id}/reviews (message)

BUSINESS IMPACT

One review reaches every customer who opens that product. Because the session token is readable from browser storage rather than being held in an HttpOnly cookie, the payload can take over the accounts of everyone who views the page, staff included.

LIKELIHOOD

High. Any registered customer can post a review, and no moderation step exists.

ATTACKER PREREQUISITES

A registered account.

TECHNICAL DESCRIPTION

The review body is stored without encoding and rendered into the product page as markup. A review containing an image tag with an error handler executes for every visitor to that product, including staff using the administration view of the same content.

REPRODUCTION

1. Sign in as any customer.
2. Open any product and submit a review whose body is: `<img src=x onerror="document.title=document.title+' XSS'">`
3. Open the same product page in a second browser signed in as a different customer.
4. Observe that the injected handler runs, confirmed by the modified document title.

EVIDENCE

TOOL OUTPUT, REDACTED

```
$ attestor run --check ATT-2026-000-004
reproduced 1/1 attempts
evidence captured: 1 object
```

REMEDIATION

Encode the review body for the HTML context at render time rather than filtering it at submission, and render it as text unless rich content is a product requirement. If it is, run the submitted markup through an allowlist sanitiser on the server and serve a Content-Security-Policy that does not permit inline event handlers. Moving the session token out of browser storage into a Secure, HttpOnly, SameSite cookie removes the account-takeover consequence even where an injection is later missed.

REFERENCES

- [OWASP Cheat Sheet: Cross Site Scripting Prevention](https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html)
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

ATT-2026-000-005

Arbitrary file read through the document download endpoint

HIGH

8.7

CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

CWE-22

A01:2025

WSTG-ATHZ-01

ASVS 5.1.1

AFFECTED

- GET juice.attestor-lab.internal/ftp (file)

BUSINESS IMPACT

Application configuration, environment files and the SQLite database file are all readable without authentication. The configuration contains the token signing material, which turns this single issue into a complete authentication bypass without needing the separate token weakness above.

LIKELIHOOD

High. Unauthenticated, and the endpoint is linked from the application itself.

ATTACKER PREREQUISITES

None.

TECHNICAL DESCRIPTION

The file download endpoint resolves the requested name relative to a directory without normalising it first. Traversal sequences, including their URL-encoded and double-encoded forms, escape the directory and return any file readable by the application process.

REPRODUCTION

1. Request GET /ftp/quarantine/%2e%2e%2f%2e%2e%2fpackage.json.
2. Observe the file contents in the response with a 200 status.
3. Repeat with the path of the configuration file to confirm the traversal is not limited to one directory.

EVIDENCE

APPLICATION CONFIGURATION RETURNED BY THE DOWNLOAD ENDPOINT

```
GET /ftp/quarantine/%2e%2e%2f%2e%2e%2fpackage.json HTTP/1.1
Host: juice.attestor-lab.internal

--- response ---
HTTP 200
Content-Type: application/json

{"name":"sample-retail-storefront","version":"14.2.0","dependencies":{" ... }}
```

REMEDIATION

Resolve the requested path against the intended base directory and reject it unless the resolved absolute path is still inside that directory — a prefix comparison after `path.resolve``, not a string check on the input. Decode once, and reject rather than repeatedly decoding. Serve downloads by an identifier that maps to a file server-side instead of accepting a filename from the client at all, which removes the class rather than this instance of it.

REFERENCES

- [OWASP Cheat Sheet: File Upload and Path Traversal](https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html)
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

ATT-2026-000-006

Order total accepted from the client, allowing an order to be placed for any amount

HIGH

7.1

CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

CWE-840

A06:2025

WSTG-BUSL-01

AFFECTED

- POST juice.attestor-lab.internal/rest/basket/{id}/checkout (totalPrice)

BUSINESS IMPACT

Goods can be ordered for any amount the customer chooses, including zero. The loss is direct and per-order, and because the order record itself carries the manipulated total, reconciliation against the payment provider is the only place it would be noticed.

LIKELIHOOD

High for anyone who opens the browser network tab. This class of issue is routinely found by customers, not only by attackers.

ATTACKER PREREQUISITES

A registered account and an item in the basket.

TECHNICAL DESCRIPTION

The checkout request carries the basket total as a field and the server records it without recomputing it from the basket contents. Submitting a modified total produces an order for that amount, and the coupon field is applied after the total is fixed rather than before, so a discount can be applied twice.

REPRODUCTION

1. Add a product priced above 1,000 to the basket.
2. Intercept the checkout request and change totalPrice to 1.
3. Submit the request.
4. Open the order confirmation and observe the order recorded at the submitted total.

EVIDENCE

CHECKOUT ACCEPTED AT A TOTAL SUBMITTED BY THE CLIENT

```
POST /rest/basket/1/checkout HTTP/1.1
Host: juice.attestor-lab.internal
Content-Type: application/json

{"basketId":1,"totalPrice":1,"couponData":""}

--- response ---
HTTP 200

{"orderConfirmation":{"orderId":"a1b2c3","totalPrice":1,"paymentId":"[REDACTED]"}}
```

REMEDIATION

Recompute the total on the server from the basket contents, the current catalogue prices, the applicable tax and the delivery option, and ignore any total supplied by the client. Apply coupons as part of that computation, once, and record the computed total against the payment intent so the payment provider and the order cannot disagree. Add a reconciliation check that rejects an order whose recorded total differs from the authorised payment amount.

REFERENCES

- [OWASP WSTG: Testing for Business Logic Data Validation](https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/10-Business_Logic_Testing/01-Test_Business_Logic_Data_Validation)
https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/10-Business_Logic_Testing/01-Test_Business_Logic_Data_Validation

ATT-2026-000-007

Security headers absent across the application

MEDIUM

5.1

CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:P/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N

CWE-693

A02:2025

WSTG-CONF-12

ASVS 3.4.1

AFFECTED

- juice.attestor-lab.internal/
- juice.attestor-lab.internal/rest/products/search
- juice.attestor-lab.internal/rest/user/whoami
- juice.attestor-lab.internal/api/Products
- juice.attestor-lab.internal/assets/*

BUSINESS IMPACT

No single header is an exploit on its own. Together their absence removes the layer that would have limited the impact of the stored cross-site scripting above: a policy without inline script would have prevented that payload from running at all.

LIKELIHOOD

Not directly exploitable; it increases the impact of other findings.

ATTACKER PREREQUISITES

Another finding to amplify.

TECHNICAL DESCRIPTION

No Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, Referrer-Policy or Permissions-Policy header is set on any response. This is one configuration change rather than one issue per endpoint, and it is recorded here once with every affected path listed.

REPRODUCTION

1. Request any page and inspect the response headers.
2. Observe that none of the listed headers is present.

EVIDENCE**TOOL OUTPUT, REDACTED**

```
$ attestor run --check ATT-2026-000-007
reproduced 1/1 attempts
evidence captured: 1 object
```

REMEDIATION

Set the headers at the edge so every response carries them: `Content-Security-Policy` with `default-src 'self'` and no `unsafe-inline`, `Strict-Transport-Security: max-age=31536000; includeSubDomains`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin` and a `Permissions-Policy` that disables the interfaces the application does not use. Deploy the policy in report-only mode first, collect violations for a week, then enforce.

REFERENCES

- [OWASP Cheat Sheet: HTTP Security Response Headers](https://cheatsheetseries.owasp.org/cheatsheets/HTTP-Headers_Cheat_Sheet.html)
https://cheatsheetseries.owasp.org/cheatsheets/HTTP-Headers_Cheat_Sheet.html

ATT-2026-000-009

Three dependencies with published vulnerabilities are reachable from request handling

MEDIUM	5.1	CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:L/VA:L/SC:N/SI:N/SA:N	CWE-1104	A03:2025
---------------	-----	---	----------	----------

AFFECTED

- juice.attestor-lab.internalpackage-lock.json

BUSINESS IMPACT

Two of the three are denial-of-service issues in parsing code, which matters less here than the third, a prototype pollution issue in a utility used by the request body parser and reachable from unauthenticated input.

LIKELIHOOD

Medium. Requires a specific request shape but no authentication.

ATTACKER PREREQUISITES

Ability to send a request body to the application.

TECHNICAL DESCRIPTION

The dependency tree carries three components with published vulnerabilities that are reachable from code on the request path, as distinct from the twenty-one further matches that are present only in the build tooling and are recorded in the appendix rather than reported here.

REPRODUCTION

1. Generate a bill of materials for the deployed image.
2. Correlate the resolved versions against published advisories.
3. Confirm reachability by tracing the import path from the request handler to the affected function.

EVIDENCE

TOOL OUTPUT, REDACTED

```
$ attestor run --check ATT-2026-000-009
reproduced 1/1 attempts
evidence captured: 1 object
```

REMEDIATION

Upgrade the three reachable components to the fixed versions listed in the appendix; the upgrade is a patch-level change in each case with no interface change. Add dependency scanning to the pipeline so this is caught at build time, and pin the lockfile so what is scanned is what deploys.

REFERENCES

- [OWASP Top 10 2025: A03 Software Supply Chain Failures](https://owasp.org/Top10/)
<https://owasp.org/Top10/>

ATT-2026-000-010

No rate limiting on the login endpoint

MEDIUM

6.9

CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N

CWE-307

A07:2025

API4:2023

WSTG-ATHN-03

ASVS 6.2.4

AFFECTED

- POST juice.attestor-lab.internal/rest/user/login

BUSINESS IMPACT

Credential stuffing against this application is unimpeded, and combined with the enumeration finding above the attacker knows which addresses are worth trying. For a retailer holding saved payment methods, a successful stuffing run is a fraud problem rather than only a privacy one.

LIKELIHOOD

High. Credential stuffing is automated and continuous across the internet.

ATTACKER PREREQUISITES

A credential list, which is freely available.

TECHNICAL DESCRIPTION

The login endpoint accepted 400 attempts against a disposable test account without throttling, lockout or any change in response. Testing was stopped at 400 by our own ceiling rather than by the application. No account lockout was triggered and no alert was generated.

REPRODUCTION

1. Create a disposable test account.
2. Submit repeated login attempts with an incorrect password against that account only.
3. Observe that responses remain consistent and no throttling, delay or lockout occurs.
4. Testing was stopped at 400 attempts by our own limit.

EVIDENCE

TOOL OUTPUT, REDACTED

```
$ attestor run --check ATT-2026-000-010
reproduced 1/1 attempts
evidence captured: 1 object
```

REMEDIATION

Apply rate limiting scoped to both the account and the source address, with an exponential backoff on repeated failure, and prefer a verification challenge over a hard lockout so an attacker cannot lock out real customers. Check submitted passwords against a breached-password list at registration and at change, which addresses the underlying reason stuffing works. Log and alert on failure rates per account rather than only in aggregate.

REFERENCES

- [OWASP Cheat Sheet: Credential Stuffing Prevention](https://cheatsheetseries.owasp.org/cheatsheets/Credential_Stuffing_Prevention_Cheat_Sheet.html)
https://cheatsheetseries.owasp.org/cheatsheets/Credential_Stuffing_Prevention_Cheat_Sheet.html

ATT-2026-000-008

Registered email addresses confirmed by the password reset response

LOW

5.3

CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

CWE-204

A07:2025

WSTG-IDNT-04

ASVS 6.2.3

AFFECTED

- POST `juice.attestor-lab.internal/rest/user/reset-password` (email)

BUSINESS IMPACT

On its own this is minor. In combination with the absent rate limiting on the login endpoint it turns a generic credential-stuffing list into a targeted one, which is the difference between a low success rate and a high one.

LIKELIHOOD

High as an aid to another attack; not damaging alone.

ATTACKER PREREQUISITES

A list of candidate email addresses.

TECHNICAL DESCRIPTION

The password reset endpoint returns a different message and a measurably different response time for a registered address than for an unregistered one, which allows a list of addresses to be sorted into customers and non-customers.

REPRODUCTION

1. Submit a reset request for a known registered address and record the response body and timing.
2. Submit the same request for an address that is not registered.
3. Compare: the message differs and the registered case is consistently slower.

EVIDENCE

TOOL OUTPUT, REDACTED

```
$ attestor run --check ATT-2026-000-008
reproduced 1/1 attempts
evidence captured: 1 object
```

REMEDIATION

Return the same response body, status and approximate timing in both cases — accept the request, say that an email has been sent if the address is registered, and do the work asynchronously so the timing does not vary. Apply the same treatment to registration and to the email-change flow, which have the same tell.

REFERENCES

- [OWASP Cheat Sheet: Forgot Password](https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html)
https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html

8. Positive observations

Controls we found working. A report that lists only failures gives a false picture of the system and of the people who built it.

- Passwords are stored using a modern password hashing function with per-user salts. Nothing in the storage path required change.
- TLS is correctly configured on the public listener: TLS 1.2 and 1.3 only, no weak cipher suites, and a valid certificate chain with 74 days to expiry.
- The administration area is not linked from the customer interface and is not indexed, which slowed discovery even though it did not prevent it.
- The payment provider integration uses hosted fields, so card numbers never reach the application. This is the correct design and it removed an entire class of finding from this assessment.

9. Strategic recommendations

Grouped by root cause, not by finding. Several findings often share a single fix.

First 30 days

- Parameterise every raw query in the codebase, not only the login handler. Static analysis will find the rest in an afternoon.
- Pin token verification to one algorithm, rotate the signing key, and invalidate live sessions.
- Recompute order totals server-side and add a reconciliation check against the payment provider.
- Remove the client-supplied filename from the download path.

Days 30 to 60

- Introduce an authorisation layer that resolves the object and checks ownership, and apply it to every handler that takes an identifier. The basket finding is one instance of a pattern.
- Move the session token out of browser storage into a Secure, HttpOnly, SameSite cookie.
- Deploy the security headers in report-only mode, collect violations for a week, then enforce.

Days 60 to 90

- Add rate limiting and a breached-password check on the authentication paths.
- Put dependency scanning into the pipeline and fail the build on a reachable high-severity match.
- Book the retest. One is included within 30 days of this report.

10. Compliance mapping

This mapping is provided for convenience in evidencing your own programme. It is not a compliance opinion, not an audit opinion and not a certification. Attestor Security is not empanelled by CERT-In.

ISO/IEC 27001:2022

CONTROL	TITLE	WHAT THE AUDITOR EXPECTS	RELATED FINDINGS
A.8.8	Management of technical vulnerabilities	Evidence that technical vulnerabilities are identified, evaluated and remediated, with dates.	10
A.8.9	Configuration management	Evidence that secure configuration baselines are defined and verified.	1
A.8.25	Secure development lifecycle	Evidence that security is verified before a release reaches production.	0
A.8.26	Application security requirements	Evidence that application security requirements were defined and tested against.	0
A.8.28	Secure coding	Evidence of secure coding verification, including static analysis output.	2
A.8.29	Security testing in development and acceptance	An independent test report covering the release under review.	0

CONTROL	TITLE	WHAT THE AUDITOR EXPECTS	RELATED FINDINGS
A.5.7	Threat intelligence	Evidence that known exploited vulnerabilities are tracked and acted upon.	0

SOC 2 Trust Services Criteria

CONTROL	TITLE	WHAT THE AUDITOR EXPECTS	RELATED FINDINGS
CC4.1	Monitoring of controls	Evidence of periodic evaluation of the effectiveness of controls.	0
CC6.1	Logical access security	Evidence that logical access controls restrict access to authorised users only.	2
CC6.6	Boundary protection	Evidence that external threats to the system boundary are controlled.	0
CC6.8	Unauthorised software prevention	Evidence that unauthorised or malicious software is prevented.	0
CC7.1	Vulnerability detection and monitoring	Evidence of vulnerability identification, including independent penetration testing.	10
CC7.2	Security event monitoring	Evidence that anomalies are detected and analysed.	0
CC8.1	Change management	Evidence that changes are tested and authorised before deployment.	0

PCI DSS 4.0

CONTROL	TITLE	WHAT THE AUDITOR EXPECTS	RELATED FINDINGS
6.2	Bespoke and custom software developed securely	Evidence that custom software is reviewed for vulnerabilities.	2
6.4.1	Public-facing web application protection	Evidence of an annual review of public-facing web applications by a specialised party.	0
11.3.1	Internal vulnerability scans	Quarterly internal scan reports with remediation and rescans.	0
11.4.2	Internal penetration testing	An annual internal penetration test performed by a qualified party.	0
11.4.3	External penetration testing	An annual external penetration test and after any significant infrastructure change.	10
11.4.4	Correction of exploitable vulnerabilities	Evidence that findings were corrected and testing repeated to verify.	0

Digital Personal Data Protection Act, 2023

CONTROL	TITLE	WHAT THE AUDITOR EXPECTS	RELATED FINDINGS
s8(5)	Reasonable security safeguards	Evidence that the data fiduciary takes reasonable security safeguards to prevent a personal data breach.	10
s8(4)	Processor obligations under contract	A data processing agreement and evidence that processors are held to equivalent safeguards.	0
s8(7)	Erasure on withdrawal or purpose completion	Evidence that personal data is erased when the purpose is served, including by processors.	0
s8(6)	Breach intimation	Evidence of a process to intimate the Board and affected data principals of a breach.	0

11. Appendices

Asset inventory

- juice.attestor-lab.internal – customer web application, staging
- juice.attestor-lab.internal/rest/* – REST API consumed by the web application
- juice.attestor-lab.internal/api/* – legacy REST surface, still reachable
- juice.attestor-lab.internal/#/administration – administration interface

Ports and services

HOST	PORT	SERVICE	VERSION
juice.attestor-lab.internal	443	https	nginx 1.27
juice.attestor-lab.internal	80	http	nginx 1.27, redirects to 443

Out-of-scope notes

- The payment provider was excluded by agreement and was not contacted at any point.
- No denial-of-service, load or volumetric testing was performed. The platform used for this assessment contains no such capability.

Glossary

CVSS	Common Vulnerability Scoring System. A published method for scoring the severity of a finding. The vector string beside each score records how it was derived.
IDOR	Insecure direct object reference. A request that identifies a record by an identifier the server does not check the caller is entitled to.
Attack narrative	A written walkthrough of how several findings combine into one outcome. Included because severity per finding does not describe combined risk.

12. Limitations and disclaimer

This wording is in draft and has not yet been reviewed by a qualified lawyer.

LIMITATIONS AND DISCLAIMER

Nature of this assessment. This report presents the results of a point-in-time security assessment carried out by Attestor Security, trading as Attestor Security, for Sample Retail Private Limited between 13 July 2026 and 17 July 2026, against the scope recorded in the Scope section of this report. The findings describe the state of the tested assets as observed during that period only. Systems change; a result that was accurate during the test window may cease to be accurate immediately afterwards as a consequence of code changes, configuration changes, dependency updates, infrastructure changes or newly disclosed vulnerabilities.

Scope limitation. Only the assets, environments, interfaces and user roles listed in the Scope section were assessed. Assets that were excluded from scope, that were unavailable during the test window, or for which access or credentials were not provided, were not assessed and no statement is made about them. The Coverage Matrix in this report records what was tested, what was partially tested and what was not tested, together with the reason in each case.

No warranty of completeness. Security assessment is a sampling exercise performed under time and scope constraints. No assessment, however thorough, can identify every vulnerability in a system, and the absence of a finding in this report must not be interpreted as evidence that a vulnerability does not exist. Attestor Security does not warrant that the tested assets are free from vulnerabilities, secure against all attacks, or fit for any particular purpose.

Not a certification. This report is not a certificate, accreditation, attestation of compliance, or guarantee of compliance with any law, standard, framework or contractual obligation. Where findings are mapped to a standard such as ISO/IEC 27001, SOC 2, PCI DSS or India's Digital Personal Data Protection Act, that mapping is provided for the client's convenience in evidencing its own programme and does not constitute a compliance opinion, an audit opinion, or certification of any kind. Attestor Security is not empanelled by CERT-In and this report is not a CERT-In audit report.

Methodology. The assessment followed OWASP Web Security Testing Guide 4.2, OWASP Application Security Verification Standard 5.0, OWASP Top 10:2025, OWASP API Security Top 10:2023, NIST SP 800-115, PTES. Automated tooling was used to obtain coverage and all reported findings were reviewed and validated by a human tester before inclusion. Findings arising solely from automated output and not validated by a tester are not included in this report.

Client responsibilities. Sample Retail Private Limited confirmed, prior to testing, that it owns or is otherwise entitled to authorise testing of the assets in scope, that any necessary permissions from third parties including hosting, cloud and service providers had been obtained, and that current backups existed. The accuracy of information supplied by Sample Retail Private Limited, including asset inventories, architecture descriptions and credentials, has been relied upon and has not been independently verified except where stated.

Remediation. Remediation guidance in this report is provided in good faith and should be reviewed, adapted to the client's environment and tested by Sample Retail Private Limited before deployment. Attestor Security is not responsible for the implementation of remediation, nor for any effect of implementing it. Findings recorded as resolved were re-verified only where a retest was performed and only to the extent recorded in the retest results.

Reliance and distribution. This report is prepared solely for Sample Retail Private Limited and for the internal purposes described in the engagement documents. It may be shared with the client's auditors, insurers, regulators and prospective customers where necessary for assurance purposes, on the basis that no duty of care is owed by Attestor Security to any such recipient and no such recipient may rely on this report. It must not be published, quoted in part, or used in marketing without the prior written consent of Attestor Security and

Sample Retail Private Limited.

Confidentiality. This report contains information about unresolved security weaknesses. Disclosure to unauthorised parties may materially increase the risk to Sample Retail Private Limited. It must be handled as confidential and distributed on a need-to-know basis.

Liability. Except in respect of liability which cannot lawfully be limited, the total aggregate liability of Attestor Security arising out of or in connection with this engagement, whether in contract, tort including negligence, or otherwise, is limited to the fees actually paid by Sample Retail Private Limited for this engagement. Attestor Security is not liable for indirect, special or consequential loss, nor for loss of profit, revenue, goodwill, data or anticipated savings. Nothing in this report varies the terms of the executed engagement documents, which prevail in the event of conflict.

Governing law. This report and the engagement to which it relates are governed by the laws of India and subject to the exclusive jurisdiction of the courts at the registered office of Attestor Security, unless the executed engagement documents provide otherwise.